

AKS Interview Questions and Answers

Printable study guide for Azure Kubernetes Service interviews - prepared June 2, 2026

This guide focuses on common AKS topics asked in Azure DevOps, SRE, Platform Engineering, and Cloud Infrastructure interviews. It includes production-focused answers you can rehearse directly.

Strong opening statement

In AKS, my focus is not only deploying containers. I also look at production operations, secure cluster design, networking, CI/CD integration, monitoring, autoscaling, incident troubleshooting, storage planning, and cost-aware node pool management.

AKS Core and Architecture

1. What is AKS?

AKS, or Azure Kubernetes Service, is Microsoft's managed Kubernetes service for deploying, scaling, and operating containerized applications on Azure. Azure manages much of the Kubernetes control plane responsibility, while the platform team manages workloads, node pools, networking, identity, security, upgrades, and observability.

2. What is the difference between AKS and Kubernetes?

Kubernetes is the open-source container orchestration platform. AKS is Azure's managed implementation of Kubernetes. AKS adds Azure integrations such as Azure Container Registry, Azure Monitor, Log Analytics, Microsoft Entra ID, managed identities, Azure CNI networking, Key Vault, and Azure load balancing services.

3. What are node pools in AKS?

Node pools are groups of worker nodes with the same configuration. A system node pool hosts critical cluster components, while user node pools are typically used for application workloads. Separate node pools help with scaling, workload isolation, cost control, and using different VM sizes for different workload types.

Networking, Exposure, and Scaling

4. How does networking work in AKS?

AKS networking includes pod IP assignment, node networking, services, ingress routing, DNS, private endpoints, egress control, network policies, and Azure load balancer integration. In production, I would focus on IP planning, private cluster design if needed, secure ingress, DNS resolution, and controlled egress to protect the environment.

5. How do you expose an application in AKS?

Common options include ClusterIP for internal-only service access, LoadBalancer for exposing services through an Azure load balancer, and Ingress for HTTP/HTTPS routing. In production, Ingress is commonly used with NGINX Ingress Controller, Application Gateway Ingress Controller, TLS certificates, and DNS records.

6. How do you scale workloads in AKS?

I would use Horizontal Pod Autoscaler to scale pod replicas based on CPU, memory, or custom metrics. I would also use Cluster Autoscaler so AKS can add or remove nodes when pods cannot be scheduled or when nodes are underused. Scaling should be tested with resource requests, limits, and realistic load patterns.

Security, Identity, Monitoring, and Upgrades

7. How do you secure AKS?

I would secure AKS using Microsoft Entra ID integration, Azure RBAC, least privilege, managed identities, workload identity, network policies, private clusters where required, image scanning, Key Vault integration, secure ingress, pod security standards, and regular Kubernetes and node image upgrades.

8. What is Workload Identity in AKS?

Microsoft Entra Workload ID allows Kubernetes workloads to access Azure resources without storing long-lived secrets inside pods. A pod can use a federated identity configuration to authenticate to Azure services through a managed identity style pattern. This is safer than placing client secrets or connection secrets in application configuration.

9. How do you monitor and troubleshoot AKS?

I would use Azure Monitor, Container Insights, Log Analytics, Prometheus/Grafana if enabled, Kubernetes events, pod logs, node metrics, and commands such as `kubectl get, describe, logs, top, and events`. I would also review deployment status, service endpoints, ingress configuration, DNS, image pull errors, node pressure, and failed probes.

10. How do you perform AKS upgrades safely?

I would check supported Kubernetes versions, review deprecated APIs, test in a lower environment first, use maintenance windows, upgrade node pools carefully, validate applications after the upgrade, monitor cluster health, and keep rollback or redeployment plans ready. Production upgrades should be planned, tested, and communicated.

Scheduling, Health Probes, and Workload Types

11. What is pod affinity and anti-affinity in AKS?

Affinity controls where pods should be scheduled. Node affinity can place pods on selected node types, such as GPU nodes, memory-optimized nodes, or specific user node pools. Pod affinity can place related pods near each other. Pod anti-affinity spreads pods apart across nodes or zones to improve availability. In AKS, I would use anti-affinity or topology spread constraints to avoid placing all replicas on the same node or failure domain.

12. What is a readiness probe?

A readiness probe tells Kubernetes whether a pod is ready to receive traffic. If the readiness probe fails, Kubernetes keeps the pod running but removes it from service load balancing. This helps avoid sending traffic to an application that is still starting, warming up, waiting for dependencies, or temporarily unable to serve requests.

13. What is the difference between liveness and readiness probes?

A liveness probe checks whether the container should be restarted. A readiness probe checks whether the pod should receive traffic. For example, if an application is completely hung, the liveness probe can restart it. If the application is running but temporarily unable to connect to a backend, the readiness probe can keep traffic away until it recovers.

14. What is the difference between stateful and stateless services in AKS?

A stateless service does not store important state inside the pod. If the pod is deleted, another pod can replace it easily. Web APIs and frontend services are common examples and are usually deployed with Deployments. A stateful service needs stable identity, persistent storage, or ordered startup and shutdown. Databases, message brokers, and applications needing persistent volumes are common examples and are often deployed with StatefulSets.

Storage and Kubernetes Controllers

15. How does persistent storage work across Availability Zones in AKS?

AKS uses Persistent Volumes, Persistent Volume Claims, StorageClasses, and CSI drivers such as Azure Disk CSI and Azure Files CSI. In multi-zone clusters, storage design is important because some disks can be tied to a specific zone. If a pod is rescheduled to another zone, a zone-bound disk can limit recovery. For higher availability, I would review whether the workload needs zone-redundant storage, Azure Files, backup/restore, replication, or application-level high availability. For stateless workloads, I mainly spread replicas across zones. For stateful workloads, I also validate where the data lives and how it recovers during a zone failure.

16. What is a DaemonSet in AKS?

A DaemonSet ensures that a copy of a pod runs on every eligible node, or on selected nodes based on node selectors, taints, tolerations, or affinity rules. In AKS, DaemonSets are commonly used for node-level agents such as log collectors, monitoring agents, security agents, network plugins, and storage drivers. When a new node is added, the DaemonSet automatically schedules the required pod on that node.

17. What is a ReplicaSet in AKS?

A ReplicaSet ensures that a specified number of pod replicas are running at all times. If the desired count is three and one pod fails, the ReplicaSet creates another pod to return to the desired count. In production, teams usually do not create ReplicaSets directly. They define a Deployment, and the Deployment manages ReplicaSets during rollout, rollback, scaling, and updates.

Quick Comparison Table

Concept	Main purpose	Common AKS use case
Deployment	Manages ReplicaSets and rolling updates	Most stateless production applications
ReplicaSet	Keeps a fixed number of pod replicas running	Application pod replicas managed by a Deployment
DaemonSet	Runs one pod on every eligible node	Logging, monitoring, networking, security, storage agents
StatefulSet	Manages stateful pods with stable identity/storage	Databases, brokers, apps needing persistent storage
Readiness probe	Controls whether a pod receives traffic	Avoid routing users to pods that are not ready
Liveness probe	Controls whether a container should be restarted	Self-healing for hung or unhealthy processes

Production-focused closing answer

For AKS production support, I validate the full path: image pull, pod scheduling, probes, service endpoints, ingress, DNS, network policy, identity, secrets, persistent storage, node health, autoscaling behavior, and observability. That helps me troubleshoot issues systematically instead of only checking whether pods are running.

References

Microsoft Learn - AKS core concepts: <https://learn.microsoft.com/en-us/azure/aks/core-aks-concepts>
Microsoft Learn - AKS storage concepts: <https://learn.microsoft.com/en-us/azure/aks/concepts-storage>
Microsoft Learn - AKS best practices: <https://learn.microsoft.com/en-us/azure/aks/best-practices>
Microsoft Learn - Cluster autoscaler in AKS: <https://learn.microsoft.com/en-us/azure/aks/cluster-autoscaler>
Microsoft Learn - Workload identity in AKS: <https://learn.microsoft.com/en-us/azure/aks/workload-identity-overview>
Microsoft Learn - Diagnose and solve AKS problems: <https://learn.microsoft.com/en-us/azure/aks/aks-diagnostics>
Kubernetes Docs - Assigning pods to nodes: <https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/>

Kubernetes Docs - Probes: <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes>
Kubernetes Docs - StatefulSet: <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>
Kubernetes Docs - DaemonSet: <https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/>
Kubernetes Docs - ReplicaSet: <https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>